How Roblox Gives AI Agents Access Without Giving Them Credentials

John Judge, Shounak Datta, Ahmad Alomari, Nicole Grinstead
Roblox
{jjudge, sdatta, aalomari, ngrinstead}@roblox.com

Abstract

AI agents that work on an engineer's behalf may need to call a number of APIs. The simplest way to allow this is to give each agent long-lived API keys, but that puts those keys inside a process we cannot fully trust: a single prompt injection could leak them, and each additional key increases the damage a leak can cause.

This paper describes the approach Roblox uses instead. Each agent receives a short-lived identity tied to the engineer it works for. A *Credential Broker* stores the engineer's vendor credentials and trades the agent's identity for short-lived access tokens. An *MCP Gateway* then attaches those tokens to the agent's outgoing requests, so in most cases the agent never sees a credential at all. The system currently handles traffic to more than 50 backend services and issues roughly 8,400 tokens per day, without any agent holding a long-lived credential.

Keywords: AI agents, agent security, Model Context Protocol, credential broker, credential injection, OAuth, token exchange, workload identity, prompt injection, least privilege

1 Introduction

In some instances, engineers at Roblox increasingly leverage AI agents to manage certain tasks. This can include, for example: reading from wikis, filing and updating issues, querying observability backends, and pushing code. To be useful, an agent has to authenticate to dozens of third-party services on behalf of the engineer running it.

The obvious approach is to hand each agent a set of API keys, one per backend service. But extending that approach to more keys and more services expands the potential blast radius of any leaked credential. In this paper, we walk through the system we built instead: short-lived, human-attributed identity plus a credential broker. Today that system mediates traffic to more than 50 backend services and vends roughly 8,400 tokens per day across our sandbox runtimes, and no agent ever holds a long-lived credential.

Why handing agents tokens doesn't work. Static API keys carry two problems on their own. An agent makes both worse.

- **No one knows they exist, so no one can revoke them.** Many API keys have no central inventory. When a laptop is lost or an engineer leaves, no one can say which tokens are still valid or where they were copied.
- **They're almost always over-scoped.** An engineer's keys typically grant read and write across dozens of services. An agent handed those tokens inherits the whole bundle, even if its task needs one read-only endpoint.
- **An agent multiplies both.** A key in a human's keychain gets used a few times a day by someone who knows what they're doing. The same key in an agent's context is one prompt injection away from being sent to an attacker [2, 7], and it sits in a process that will happily use every scope it has [4].

At Roblox, agents reach tools through two paths: the Model Context Protocol (MCP) [1], and Skills, which are tool-specific scripts that read credentials from their environment. Both need the same guarantee: an agent has to authenticate to an upstream service without ever holding a long-lived credential.

Contributions of this paper include:

- **Human-attributed agent identity:** an Agent Identity layer that issues identity tokens, each one attributable to the human the agent is working for, across our sandbox runtimes.
- **Credential Broker:** a service that runs the OAuth and API key flows with each vendor and exchanges agent identity for short-lived, scoped upstream credentials.
- **MCP Gateway with credential injection:** a gateway that only exposes authorized MCP servers, mediates tool traffic, and attaches credentials to outbound requests on the agent's behalf.
- **Brokering vs. injection:** criteria for deciding whether an agent should ever hold the upstream credential at all, and the residual risk each pattern leaves.

2 System Design

Our system lets agents authenticate to upstream services without ever holding a long-lived credential, by combining human-attributed identity, a credential broker, and a mediating gateway, shown in Figure 1.

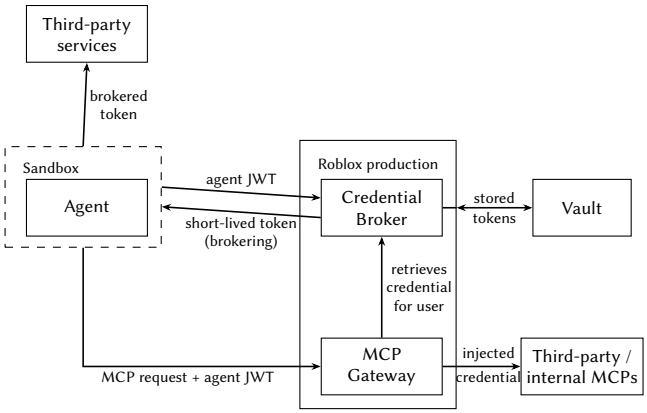


Figure 1. System overview. The agent runs in an untrusted sandbox and carries only its identity token (agent JWT). The Credential Broker holds vendor tokens in Vault and either returns a short-lived token to the agent (brokering) or hands it to the MCP Gateway, which writes it onto the outbound request (injection).

2.1 Threat Model

We treat the agent process as untrusted. Everything in the design follows from common threats to any large-scale system:

- **Prompt injection.** An attacker crafts prompts that convince an agent to exfiltrate whatever credential is sitting in its memory [2, 7].
- **Unvetted MCP servers.** An agent that can reach an arbitrary MCP server can be pointed at one an attacker controls, which then asks it for credentials or returns tool descriptions crafted to redirect what it does next.
- **Compromised agents.** A compromised agent with open access to MCP servers and credentials keeps acting until someone halts its execution.
- **Laundering actions through shared identity.** An attacker who reaches an agent running on a shared API key or machine identity acts under an identity that belongs to no single individual, and the action is unattributable to a person. Shared identity also makes conducting forensics in the event of an incident incredibly challenging.
- **Reaching further than the task requires.** Tool access is granted per agent, not per task, so an attacker who reaches an agent inherits every tool that agent can call, not just the ones the current job needs.

Five goals shape the architecture: (i) **attributable identity**, give every agent one identity that ties back to both the workload and the human; (ii) **short-lived material**, use only short-lived credential material; (iii) **containment**, keep credentials contained so they never reach the agent when they don't have to; (iv) **transparency**, stay transparent, so agents can use standard HTTP and MCP clients without a custom SDK; and (v) **least privilege**, give each agent access only to the tools it needs for the task at hand.

2.2 Architecture

Our solution has three parts:

- An **Agent Identity** layer that issues identity tokens, each one attributable to the human the agent is working for.
- A **Credential Broker** that runs the OAuth and API key flows with each vendor and exchanges agent identity for upstream credentials.
- An **MCP Gateway** that only exposes authorized MCP servers, mediates tool traffic, and attaches credentials to outbound requests on the agent's behalf.

On top of these, we use two patterns for handling credentials. In *brokering*, the agent briefly holds a scoped, short-lived vendor token. In *injection*, the gateway attaches the credential to the outbound request so the agent never sees it. Most traffic uses injection; brokering is the exception we reach for when an agent needs to speak a vendor's protocol itself.

2.3 Agent Identity

Every component in the stack needs to know two things about an incoming request: which agent this is, and which human it's acting for. Audit logs tie every action to a person, and a compromised agent can't silently act as someone else.

An agent's identity is a JWT [5] that Roblox's internal identity service issues through an OIDC token exchange [6, 9]. The token names the human on whose behalf the agent is running. How it reaches the agent depends on the runtime:

- In Roblox's managed sandbox runtime, the sandbox bootstrapping process injects the token into the agent's environment.
- In Coder workspaces, the agent exchanges the developer's GitHub session token for one.

Every consumer of that identity validates the JWT against the issuer's key set and resolves it to the human principal used for credential lookup. For calls to internal Roblox services, the broker goes a step further and exchanges the identity for a narrowly scoped token minted per request with a lifetime of about 15 minutes, so internal traffic also carries a per-request, human-attributed credential instead of a static one. Every request through the gateway logs the human principal, the agent identity, the tool called, and the upstream service, so "who did this" is answerable from one place instead of correlated across five vendors' audit logs.

2.4 Credential Broker

The Credential Broker is a microservice running in Roblox's production environment. It runs OAuth flows [3] with external vendors on behalf of the user and manages the resulting

refresh and access tokens by storing them in Vault (our system of record for secrets), then exchanges agent-identity tokens for short-lived, scoped upstream credentials.

Each (user, service) pair moves through two phases:

- **OAuth seed.** The human opens a connection URL in a browser. The broker redirects to the vendor's OAuth authorize endpoint using PKCE [8], and the user authenticates with the vendor. Vault stores the resulting tokens, tied to that user and service.
- **Token exchange.** Any agent acting on a user's behalf can request an access token by calling the broker with its identity token and a service name. No further human interaction is needed.

On every token request, the broker walks the same chain. It validates the agent's identity token, then uses that token to check the status of the (human, service) pair in Vault; if there's no connection yet, it returns the OAuth connection URL instead of a token. Otherwise it returns a scoped OAuth access token. The broker caches tokens briefly, and on a cache miss it refreshes through the vendor's refresh endpoint, persists the rotated pair, and returns it.

2.5 MCP Gateway

The MCP Gateway sits next to the broker, and it's what an agent in a sandbox actually connects to for tool calls. It exposes one logical MCP server per registered backend. From the agent's point of view, every backend looks like its own endpoint.

The gateway plays both sides of the protocol at once. To the agent, it's an MCP server: it terminates sessions, advertises tools, and returns results. To the upstream, it's an MCP client: it lists tools and forwards tool calls. The boundary in between is where credential injection happens. On each outbound tool call, the gateway asks the broker for the user's upstream credential through an endpoint only the gateway is allowed to call, then writes that credential onto the outbound request as a header. The agent never sees it.

A few design choices make the gateway practical to run:

- **Stateless backends only.** The gateway accepts only stateless Streamable-HTTP MCP servers [1]. That avoids session affinity and lets any replica serve any request behind a load balancer. In practice, this supports every MCP we integrate with.
- **Profiles.** A profile is a named collection of tools drawn from several MCP servers. A "patch third-party vulnerabilities" profile, for instance, might combine a vulnerability-lookup tool from a security data lake with a few Git tools from a source-control server. Profiles let an agent narrow its context to a task instead of loading every tool from every server, and they double as a least-privilege control: a developer who only needs the patch profile never sees the rest of the security data-lake tools.
- **Response-size caps.** Observability backends routinely return multi-megabyte payloads that overflow an agent's context window or take down a gateway replica. Past a soft cap, the gateway returns a tool-call error suggesting how to narrow the query; past a hard cap, it cuts the stream off mid-response. Either way the agent gets an actionable error instead of a silent context overflow.

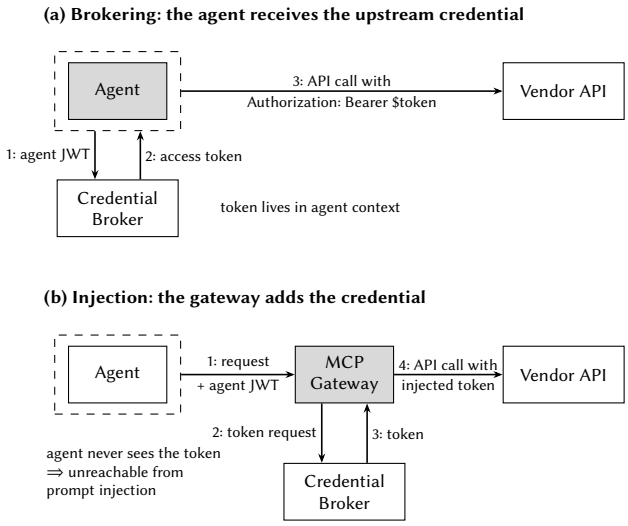


Figure 2. Brokering vs. injection. Shaded components hold the upstream credential; dashed boxes mark the untrusted sandbox. In brokering the token enters the agent's context; in injection it exists only in the gateway.

The gateway is also where content controls run. Every MCP server and tool passes security checks before the gateway exposes it, and the sandbox egress allowlist means an agent cannot reach a server the gateway doesn't front. Because every response flows back through the gateway, that is where we enforce content policy before data re-enters the agent's context: we block access to confidential documents and restricted conversations outright, and we redact attorney-client privileged material and PII. Combined with credential injection, this keeps both the credential and the sensitive data out of the agent process.

Because the gateway is the single egress point for every MCP connection, it is also where we revoke access from a runaway or compromised agent. Revoking a user's stored tokens for a service cuts off every agent acting for that user, and offboarding removes them across all services at once.

2.6 Brokering vs. Injection

The recurring question during onboarding is whether the agent should ever hold the upstream credential at all. The threat model decides. Figure 2 contrasts the two patterns.

With *credential injection*, the agent calls the gateway with only its identity token. The gateway fetches the user's credential from the broker and writes it onto the outbound request, and the agent never sees it. This is the right choice whenever the agent's only job is to invoke a tool the gateway can mediate, which covers most MCP traffic. A prompt-injection attack running inside the agent can't exfiltrate a credential that was never there. Injection is our default.

It's worth being precise about what that buys. Injection removes the credential from the agent's context, so it can't be stolen and replayed later. It does not stop a compromised agent from using a tool it legitimately has. That residual risk is then addressed with alternative controls like: egress controls, response inspection, per-request audit logging and data loss prevention tools.

With *credential brokering*, the agent calls the broker directly with its identity, gets back a vendor access token, and calls the upstream itself. This is the right choice when the agent has to speak the vendor's protocol directly: when it uses a vendor SDK, signs the request itself (as a cloud CLI does), or runs a multi-step or streaming interaction the gateway can't reasonably model. The cost is that the credential is back in the untrusted agent process, so we allow brokering only where the token is short-lived, carries no more scope than the agent needs, and the backend requires no additional gateway-side controls. Brokered traffic bypasses the gateway, so the constraint on it is at the network layer: our sandboxes can only reach hosts on an egress allowlist. A brokered token is also still a bearer token: if it leaks, it works until it expires. Short lifetimes bound that window rather than closing it; we prefer injection whenever the gateway can act as a proxy.

3 Conclusion

This system replaces long-lived personal access tokens with a broker-and-injection model built on short-lived, human-attributed identity. From here, we're extending the same pattern to all non-MCP HTTP traffic, moving the identity layer to a strongly attested SPIFFE/SPIRE based workload-identity model [10], and breaking the centralized credential service down so it can deploy alongside each sandbox. The goal stays the same: let agents do real work across every tool an engineer uses, without ever handing them a credential we can't account for.

Acknowledgments

Special thanks to Rex Belli, Harshit Kumar, Rahul Toppur, Jaskaran Singh, and Ashwin Sidhalinganahalli for their contributions to this work.

References

[1] Anthropic. 2025. Model Context Protocol Specification. https://modelcontextprotocol.io/specification/2025-06-18. Version 2025-06-18.

[2] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. 79–90. doi:10.1145/3605764.3623985

[3] Dick Hardt. 2012. The OAuth 2.0 Authorization Framework. RFC 6749. doi:10.17487/RFC6749

[4] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38.

[5] Michael B. Jones, John Bradley, and Nat Sakimura. 2015. JSON Web Token (JWT). RFC 7519. doi:10.17487/RFC7519

[6] Michael B. Jones, Anthony Nadalin, Brian Campbell, John Bradley, and Chuck Mortimore. 2020. OAuth 2.0 Token Exchange. RFC 8693. doi:10.17487/RFC8693

[7] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques for Language Models. *arXiv preprint arXiv:2211.09527* (2022).

[8] Nat Sakimura, John Bradley, and Naveen Agarwal. 2015. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636. doi:10.17487/RFC7636

[9] Nat Sakimura, John Bradley, Michael B. Jones, Breno de Medeiros, and Chuck Mortimore. 2014. OpenID Connect Core 1.0. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html.

[10] SPIFFE. 2026. SPIFFE Specifications and Concepts (SVID, Workload API). spiffe.io documentation. Online spec.